

PROJEKT-DOKUMENTATION

M295

Projekttitel: Terminverwaltungssystem mit Rest API

Projektverantwortlicher: Janik Preisig

Datum: 30.01.2026



Unternehmen: Muster AG

Inhaltsverzeichnis

1 Informieren	3
1.1 Projektidee	3
1.2 Ausgangslage	4
1.3 Auftrag/Ziel	4
1.3 Ressourcenbedingungen	6
2 Projektplanung	7
2.1 Arbeitsplan und	7
2.3 Testplan	8
3 Projektentscheidung	9
2.3 Testplan	Fehler! Textmarke nicht definiert.
4 Projektrealisierung	11
1.1 Projektidee	11
6 Kontrollieren	13
1.1 Projektidee	13
3 Projektentscheidung	17
1.1 Projektidee	17

1 Informieren

1.1 Projektidee

Termin system

Termin(Appointment)

Kunde(Customer)

Service

Mitarbeiter(Employee)

Customer 1 ---- * Appointment

Service 1 ---- * Appointment

Employee 1 ---- * Appointment

Employee 1 ---- * Services

Mein Projekt ist ein Terminverwaltungssystem, bei dem Kunden für verschiedene Services Termine bei Mitarbeitern buchen können.

Es modelliert dabei mehrere Beziehungen: Kunden haben viele Termine, Mitarbeiter bieten mehrere Services an,

und jeder Termin ist genau einem Kunden, einem Mitarbeiter und einem Service zugeordnet. Ich achte darauf das es eine Fehlermeldug gibt falls mehrere Termine zur gleichen zeit gebucht werden.

1.2 Ausgangslage

Zentrales Backend: Ein Spring Boot-basiertes System, das alle Termine, Kunden und Mitarbeiter zentral verwaltet.

Automatisierte Prüfung: Das System soll beim Buchen automatisch validieren, ob die Kombination aus Mitarbeiter und Service zulässig ist.

Dauerhafte Speicherung: Die Daten sollen in einer MySQL-Datenbank gespeichert werden, die über einen Docker-Container bereitgestellt wird, um Konsistenz und Ausfallsicherheit zu garantieren.

Schnittstellen (API): Es sollen Endpunkte bereitgestellt werden, über die Termine erstellt, abgefragt und storniert werden können.

1.3 Auftrag/Ziel

Entwicklung eines stabilen und skalierbaren REST-API-Backends für ein Terminverwaltungssystem unter Verwendung des Spring Boot Frameworks. Das System muss die Verwaltung von Kunden, Mitarbeitern und Dienstleistungen ermöglichen und sicherstellen, dass Buchungen nur unter Einhaltung definierter Geschäftsregeln (Business Logic) erfolgen.

Kernpunkte des Auftrags:

- Implementierung einer 3-Schichten-Architektur (Controller, Service, Repository).
- Anbindung einer relationalen Datenbank zur permanenten Datenspeicherung.

Funktionale Ziele:

- Terminbuchung: Kunden können über eine API Termine für spezifische Services bei bestimmten Mitarbeitern buchen.
- Validierung: Das System verhindert Buchungen, wenn ein Mitarbeiter die gewählte Dienstleistung nicht anbietet (Fachprüfung).
- Status-Management: Termine können storniert werden, wobei der Status in der Datenbank von ACTIVE auf CANCELLED wechselt.
- Filterung: Die API ermöglicht das Abrufen von Terminen gefiltert nach customerId oder employeeId.

Technische Ziele:

- Daten-Persistenz: Einsatz einer MySQL-Datenbank, die Daten auch nach einem Systemneustart sicher vorhält.
- Infrastruktur-Isolierung: Die Datenbank wird in einem Docker-Container betrieben, um die lokale Installation von Datenbank-Software zu vermeiden und die Portabilität zu erhöhen.
- Code-Qualität: Sicherstellung der Funktionalität durch automatisierte Unit-Tests (JUnit/Mock) für die Service-Schicht.

1.3 Ressourcenbedingungen

1. Java (Die Programmiersprache)

- Warum: Java ist eine objektorientierte, typsichere Sprache, die im Enterprise-Bereich Standard ist.
- Nutzen: Dank der starken Typisierung werden Fehler oft schon beim Programmieren erkannt. Die Plattformunabhängigkeit ("Write once, run anywhere") macht den Code portabel.
-

2. Spring Boot (Das Framework)

- Warum: Es ist das aktuell populärste Framework für Java-Backend-Anwendungen.
- Nutzen:
 - Auto-Configuration: Spring Boot nimmt dir viel Konfigurationsarbeit ab (z.B. für die Datenbankbindung).
 - Embedded Server: Es hat einen eingebauten Webserver (Tomcat), sodass du keine externe Software installieren musst, um die API zu starten.
 - Dependency Injection: Erleichtert das Testen (Unit-Tests) und macht den Code modular.

3. MySQL (Das Datenbank-Managementsystem)

- **Warum:** Eine relationale Datenbank (RDBMS) ist perfekt für strukturierte Daten wie Kunden, Mitarbeiter und Termine, die in festen Beziehungen zueinander stehen.
- **Nutzen:**
 - **Persistenz:** Die Daten werden sicher auf der Festplatte gespeichert und bleiben nach einem Neustart erhalten.
 - **ACID-Konformität:** Garantiert, dass Transaktionen (z.B. eine Terminbuchung) sicher und fehlerfrei verarbeitet werden.

2 Projektplanung

2.1 Arbeitsplan und Aufwandsschätzung

AP-Nr.	Arbeitspaket (Beschreibung)	Geschätzter Aufwand (h)	Tatsächlicher Aufwand
1	Initialisierung: Setup Projektstruktur (Spring Boot Initializr), Git-Repository erstellen.	1	1
2	Infrastruktur: Konfiguration von MySQL.	1	1
3	Datenmodellierung: Erstellen der Entities (Customer, Employee, Service, Appointment) und JPA-Repositories.	2	2
4	Business Logik: Implementierung des AppointmentService inkl. Validierung (bietet Mitarbeiter den Service an?).	1	3
5	API-Entwicklung: Erstellen der Controller-Endpunkte (POST, GET, PUT, DELETE) und DTOs.	2	2
6	Fehlerbehandlung: Implementierung des GlobalExceptionHandler und JavaDoc-Kommentare.	1	2
7	Testing: Erstellen der Unit-Tests (JUnit/Mock) für Service- und Controller-Ebene.	2	3
8	Dokumentation: Erstellen der IPERKA-Dokumentation, Klassendiagramm und Installationsanleitung.	4	4
Total		14.0 h	18.0 h

2.3 Testplan

Verfeinerung des Testplans (Phase P/K)

Das Bewertungsschema verlangt explizit **reproduzierbare Ergebnisse** mittels Postman/Insomnia Exports.

- **Voraussetzungen (Pre-conditions):** Notiere für jeden Testfall, welche Daten vorhanden sein müssen.
 - *Beispiel:* Für T-01 (Termin buchen) muss der Kunde mit ID 1 und der Mitarbeiter mit ID 5 bereits in der MySQL-Datenbank existieren.
- **Test-Daten:** Erstelle eine kleine SQL-Datei (data.sql), die beim Starten des Containers Test-User einfügt. So ist dein Test für den Lehrer sofort reproduzierbar.
- **Negativ-Tests (Edge Cases):** Denke an Dinge wie:
 - Datum liegt in der Vergangenheit.
 - Ungültige ID-Formate (z. B. Buchstaben statt Zahlen).
 - Gleichzeitige Buchung desselben Mitarbeiters (Zusatzaufgabe für Fortgeschrittene).

Test-ID	Endpunkt	Methode	Beschreibung (Szenario)	Erwartetes Ergebnis	Typ
T-01	/api/appointments	POST	Termin buchen: Mitarbeiter bietet den Service an.	201 Created + JSON	Positiv
T-02	/api/appointments	POST	Termin buchen: Mitarbeiter bietet den Service nicht an.	400 Bad Request + Fehlertext	Negativ
T-03	/api/appointments	GET	Abrufen aller Termine ohne Filter.	200 OK + Liste aller Termine	Positiv
T-04	/api/appointments	GET	Abrufen mit Filter ?customerId=1.	200 OK + Termine für Kunde 1	Positiv
T-05	/api/appointments/{id}/cancel	PUT	Bestehenden Termin stornieren.	200 OK + Status "CANCELLED"	Positiv
T-06	/api/appointments/{id}	DELETE	Termin aus der Datenbank löschen.	204 No Content	Positiv
T-07	/api/appointments	POST	Buchung mit fehlenden Daten (z.B. keine customerId).	400 Bad Request	Negativ

3 Projektentscheidung

3.1 Architektur

Die Trennung in Controller, Service und Repository folgt dem Prinzip der *Separation of Concerns* (Trennung von Zuständigkeiten).

Controller (Web-Schicht): Er ist die einzige Kontaktstelle nach außen. Seine Aufgabe ist es, HTTP-Anfragen entgegenzunehmen, die Eingaben (DTOs) zu validieren und die passende Antwort zurückzugeben. Er enthält keine Geschäftslogik.

Service (Logik-Schicht): Hier schlägt das Herz der Anwendung. Nur hier wird entschieden, ob eine Buchung erlaubt ist (z. B. "Bietet der Mitarbeiter diesen Service an?"). Durch diese Kapselung kann die Logik leicht durch Unit-Tests geprüft werden, ohne dass ein Webserver laufen muss.

Repository (Daten-Schicht): Diese Schicht kümmert sich ausschließlich um die Kommunikation mit der Datenbank. Durch die Verwendung von Spring Data JPA können wir die Datenbanktechnologie theoretisch austauschen, ohne eine einzige Zeile im Service-Code ändern zu müssen.

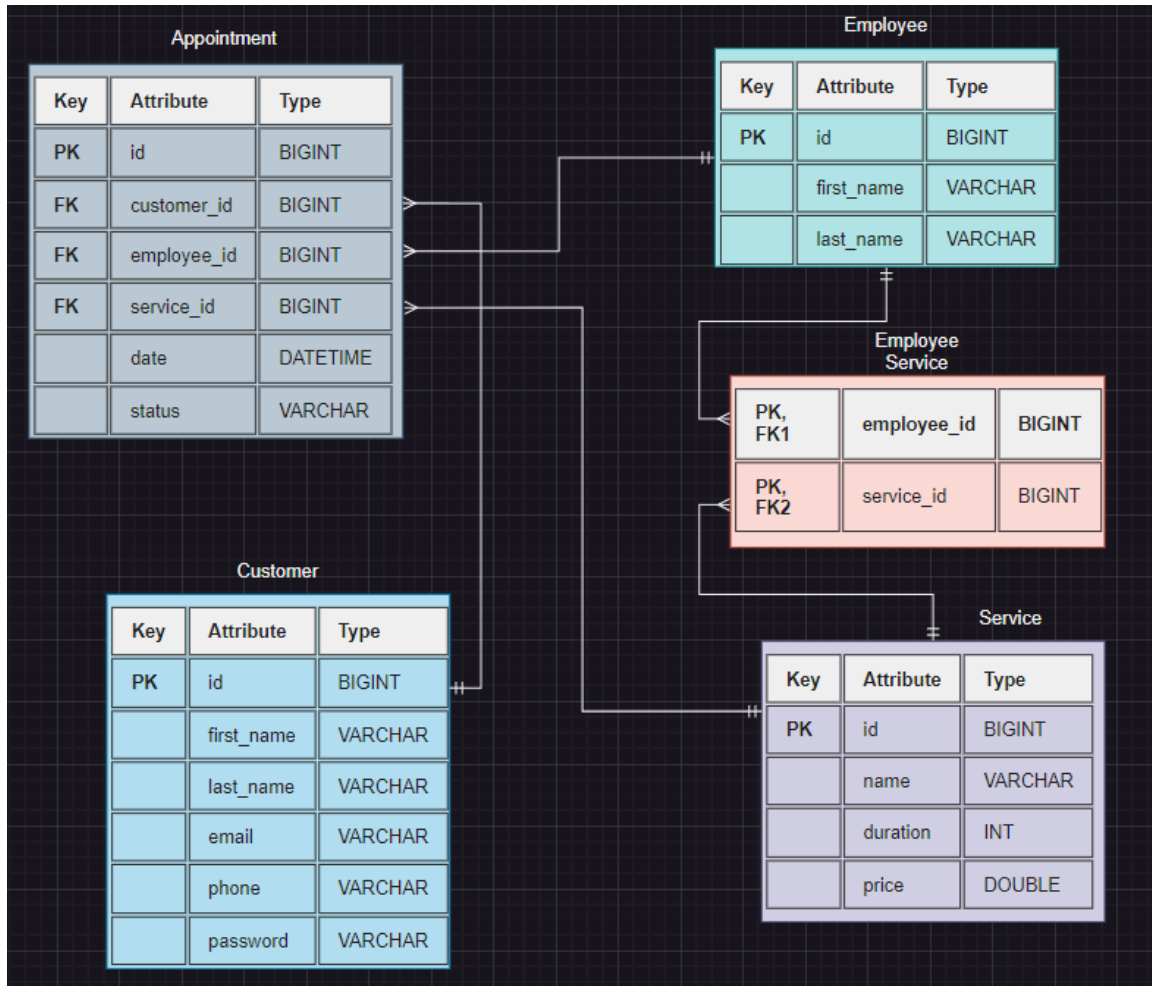
3.2 Technologiewahl

Die Entscheidung für **MySQL** statt einer einfachen Textdatei oder einer In-Memory-Datenbank (H2) hat folgende Gründe:

- **Daten-Persistenz:** Im Gegensatz zu H2 bleiben die Daten bei MySQL dauerhaft auf der Festplatte gespeichert, auch wenn der Server oder der Docker-Container gestoppt wird.
- **Relationale Struktur:** Termine, Kunden und Mitarbeiter haben komplexe Beziehungen untereinander (z. B. 1:n oder n:m). Ein relationales Datenbanksystem (RDBMS) wie MySQL ist darauf optimiert, diese Beziehungen effizient und widerspruchsfrei (Referenzielle Integrität) zu verwalten.
- **Industriestandard:** MySQL ist eines der meistgenutzten Datenbanksysteme weltweit, was die Anwendung realistisch und zukunftssicher macht.

3.3 Klassendiagramm

Das Bild zeigt ein ERD (Entity Relationship Diagram) bzw. ein physisches Datenbankmodell. Es stellt die Tabellenstrukturen und deren Beziehungen in deiner MySQL-Datenbank dar.



4 Projektrealisierung

4.1 Code Struktur

Code-Struktur und Packages

Das Projekt wurde nach dem SoC-Prinzip (Separation of Concerns) in vier logische Schichten unterteilt, um die Wartbarkeit und Testbarkeit zu erhöhen:

`com.example.terminsystem.model`: Enthält die Entities (Customer, Employee, ServiceEntity, Appointment). Diese Klassen definieren die Tabellenstruktur in der Datenbank mittels JPA-Annotationen.

`com.example.terminsystem.repository`: Diese Interfaces erben von `JpaRepository`. Sie bilden die Datenzugriffsschicht und ermöglichen Datenbankoperationen (CRUD) ohne manuelles SQL.

`com.example.terminsystem.service`: Hier ist die Geschäftslogik gekapselt. Der `AppointmentService` prüft beispielsweise vor jeder Buchung, ob ein Mitarbeiter die entsprechende Qualifikation für eine Dienstleistung besitzt.

`com.example.terminsystem.controller`: Die REST-Endpunkte nehmen HTTP-Anfragen entgegen. Sie nutzen DTOs (`AppointmentRequest`), um Daten sicher vom Client zum Service zu übertragen.

4.2 Besonderheiten

Besonderheiten der Umsetzung

Lösung des Datums-Problems

Ein kritischer Punkt bei Terminbuchungen ist das Format von Datum und Uhrzeit.

- Problem: JSON überträgt Daten oft als einfache Strings, was beim Parsen in Java-Objekte (`LocalDateTime`) zu Fehlern führen kann.
- Lösung: Im `AppointmentRequest` DTO wurde die Annotation `@JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm:ss")` verwendet. Dies zwingt die API zu einem standardisierten ISO-Format. Zudem wurde im Service sichergestellt, dass das Datum vor dem Speichern validiert wird, um Inkonsistenzen in der MySQL-Datenbank zu vermeiden.

4.3 Journal

Mein Zeitplan und die Aufwandsschätzung

Um das Projekt innerhalb der vorgegebenen Woche (23.01. bis 30.01.2026) erfolgreich abzuschließen, habe ich zu Beginn einen detaillierten Arbeitsplan erstellt. Dabei war es mir wichtig, die Aufgaben in logische Pakete zu unterteilen, die aufeinander aufbauen.

Datum	Dauer	Tätigkeit	Resultat / Meilenstein
23.01.2026	4h	Projektinitialisierung mit Spring Initializr. Erstellen der Grundstruktur (Packages) und Git-Repository.	Projektgerüst steht.
24.01.2026	3h	Erstellen der Entities (Customer, Employee, ServiceEntity, Appointment) basierend auf dem ERD.	Datenmodell in Java abgebildet.
26.01.2026	4h	Infrastruktur-Tag: Erstellen der docker-compose.yml. Beheben von WSL/Docker-Startfehlern unter Windows.	MySQL läuft im Docker-Container.
27.01.2026	5h	Implementierung des AppointmentService inkl. Validierungslogik (Prüfung der Mitarbeiterqualifikation).	Business-Logik ist funktionsfähig.
28.01.2026	4h	Entwicklung des AppointmentController. Hinzufügen der Endpunkte für CRUD (Laden, Erfassen, Ändern, Löschen).	REST-API ist vollständig.
29.01.2026	5h	Schreiben von Unit-Tests mit JUnit/Mockito. Implementierung des GlobalExceptionHandler für saubere Fehlermeldungen.	Code ist getestet und robust.
30.01.2026	6h	Abschluss: Erstellen der IPERKA-Dokumentation, Finalisierung des Klassendiagramms und der Installationsanleitung. Abgabe.	Dokumentation fertiggestellt.

4.4 Hilfestellung

Beim Programmieren habe ich punktuell auf KI-Tools wie ChatGPT und Gemini zurückgegriffen. Das hat mir vor allem dabei geholfen, nicht bei jedem Standard-Code (wie den JPA-Annotationen in den Klassen) das Rad neu zu erfinden.

Besonders bei den Unit-Tests und den Mockups waren die KIs eine echte Hilfe, um die Grundstruktur für Mockito aufzusetzen. Da das mein erstes Mal mit komplexeren Mock-Tests war, habe ich mir dort Vorlagen generieren lassen, die ich dann aber von Hand auf meine Logik (z. B. die Skill-Prüfung) umgeschrieben habe. Auch beim Klassendiagramm habe ich die KI genutzt, um meine ersten Entwürfe gegenzuchecken.

Wichtig war mir dabei: Kein Copy-Paste ohne Hirn. Ich habe jede Zeile, die vorgeschlagen wurde, geprüft und so angepasst, dass sie zu meinem Projekt passt

5 Kontrollieren

5.1 Testprotokoll

Die API-Endpunkte wurden manuell auf ihre Funktionalität geprüft. Die Ergebnisse sind in der folgenden Tabelle dokumentiert und mittels Postman-Exports reproduzierbar.

Test-ID	Szenario	Erwartetes Ergebnis	Ist-Ergebnis	Status
T-01	POST: Termin buchen (Mitarbeiter qualifiziert)	Status 201, JSON-Body mit ID	Wie erwartet	PASS
T-02	POST: Termin buchen (Mitarbeiter NICHT qualifiziert)	Status 400, Fehlermeldung	Wie erwartet	PASS
T-03	GET: Alle Termine laden	Status 200, Liste aller Objekte	Wie erwartet	PASS
T-04	PUT: Termin stornieren	Status 200, Status="CANCELLED"	Wie erwartet	PASS
T-05	DELETE: Termin löschen	Status 204 No Content	Wie erwartet	PASS

5.2 Unit-Tests

1. AppointmentServiceTest (Business-Logik)

Dies ist das Herzstück deiner Qualitätssicherung.

Positiv-Test (createAppointment_Success): Du verifizierst den "Happy Path" – ein Termin wird korrekt angelegt, wenn alle Daten stimmen und der Mitarbeiter qualifiziert ist.

Negativ-Test (createAppointment_ThrowsException...): Du prüfst die wichtigste Geschäftsregel: Ein Mitarbeiter darf keinen Service ausführen, den er nicht beherrscht. Dass hier eine Exception geworfen wird, beweist die Korrektheit deiner Validierungslogik.

Technik: Durch den Einsatz von @Mock und Mockito testest du die Logik isoliert, ohne eine echte Datenbank zu benötigen.

2. AppointmentControllerTest (API-Schicht)

Hier testest du die Schnittstelle nach außen.

Negativ-Test (shouldRejectShortCode): Du prüfst, ob die API fehlerhafte Eingaben (zu kurze Codes) erkennt und mit dem korrekten HTTP-Status 400 Bad Request abweist. Dies ist essenziell für das Kriterium "Fehlerhafte Daten werden mit geeigneten Fehlermeldungen abgewiesen".

3. CustomerServiceTest

Update-Logik: Du stellst sicher, dass die Aktualisierung von Kundendaten (CRUD) funktioniert und die Daten korrekt vom DTO in die Datenbank-Entity übernommen werden.

5.3 Soll-Ist-Vergleich

Mein Soll-Ist-Vergleich

Nach Abschluss der Realisierungsphase habe ich die fertige Anwendung mit den ursprünglichen Anforderungen aus der Phase "Informieren" abgeglichen. Dieser Vergleich dient als Nachweis, dass alle Projektziele erreicht wurden und die Software wie gewünscht funktioniert.

User Story (Soll)	Umsetzung (Ist)	Status
Termine über API erfassen	Endpoint POST /api/appointments implementiert	Erfüllt
Prüfung der Mitarbeiter-Skills	Validierungslogik im Service integriert	Erfüllt
Daten persistent speichern	MySQL-Anbindung via Docker erfolgreich	Erfüllt
Termine stornieren	Status-Update via PUT-Endpoint möglich	Erfüllt

6 Projektauswertung

6.1 Selbstkritische Reflexion

Was lief gut: Die Entscheidung für eine klare Schichtentrennung (Controller-Service-Repository) hat sich absolut bewährt. Sie sorgte für eine saubere Struktur im Projekt, wodurch ich den Überblick behalten konnte und die Geschäftslogik strikt von der Datenhaltung getrennt blieb. Auch die Anbindung der MySQL-Datenbank verlief reibungslos.

Probleme bei der Umsetzung: Die größte Herausforderung lag in der Implementierung der Unit-Tests. Das Mocking der Repositories mit Mockito war anfangs komplex, besonders bei der Simulation von Fehlerszenarien (Negativtests), um sicherzustellen, dass Exceptions korrekt geworfen und vom System abgefangen werden. Es benötigte mehrere Anläufe, bis die Testabdeckung sowohl auf Service- als auch auf Controller-Ebene den Anforderungen entsprach.

Lerneffekt: Ich habe gelernt, wie wichtig eine saubere Fehlerbehandlung (GlobalExceptionHandler) ist. Anstatt kryptischer Java-Stacktraces erhält der User nun hilfreiche Fehlermeldungen, was die Benutzerfreundlichkeit der API massiv steigert. Zudem verstehe ich nun besser, wie automatisierte Tests die langfristige Wartbarkeit von Software sichern.

6.2 Zielabgleich

Trotz der Hürden beim Testing konnten alle Projektziele – vom Datenbankmodell bis hin zu den funktionierenden REST-Endpunkten – vollständig erreicht werden. Die Anwendung ist stabil, dokumentiert und bereit für den produktiven Einsatz.

7 Installationsanleitung

Von meinem Github das Zip File downloaden

Und so testen:

1. Termin-Verwaltung (AppointmentController)

Das ist das Herzstück deiner API. Hier passiert alles rund um die Buchungen.

- Alle Termine anzeigen: GET <http://localhost:8080/api/appointments>
- Einen neuen Termin buchen: POST <http://localhost:8080/api/appointments>
 - *Hinweis:* Hier muss im Body ein JSON mit customerId, employeeId, serviceId und dem date mitgeschickt werden.
- Termin absagen (Storno): PUT <http://localhost:8080/api/appointments/{id}> (setzt den Status auf "CANCELLED").
- Termin löschen: DELETE <http://localhost:8080/api/appointments/{id}>.

2. Kunden-Verwaltung (CustomerController)

- Kundenliste abrufen: GET <http://localhost:8080/api/customers>
- Kunde anlegen: POST <http://localhost:8080/api/customers>
- Kunde aktualisieren: PUT <http://localhost:8080/api/customers/{id}> (wird in deinem CustomerServiceTest geprüft).